

 **PRODUCTION AI**

Shadow Deployment and Canary Testing

Test your new AI model against real production traffic without exposing users to risk. Find failures before they find your users.

Two complementary strategies for safe AI deployment



SHADOW MODE

% CANARY ROLLOUT



LLM-AS-JUDGE



REDIS ROLLBACK

FastAPI + asyncio + Redis. Real production architecture.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**

Why You Cannot A/B Test AI Naively

With web features, A/B testing is simple: route 50% to version A and 50% to B. With AI, naive A/B causes real user harm. Model B might hallucinate, confuse users, or give dangerous advice to real people before you notice.

👁️ Shadow Deployment

Both models run for every request. Only the production model's response is returned to the user. The challenger model's response is logged silently for comparison. Zero user risk.

📊 Canary Deployment

After shadow validation, route a small slice (1 to 5%) of real traffic to the new model. Detect live problems at minimal blast radius. Ramp gradually as confidence grows.

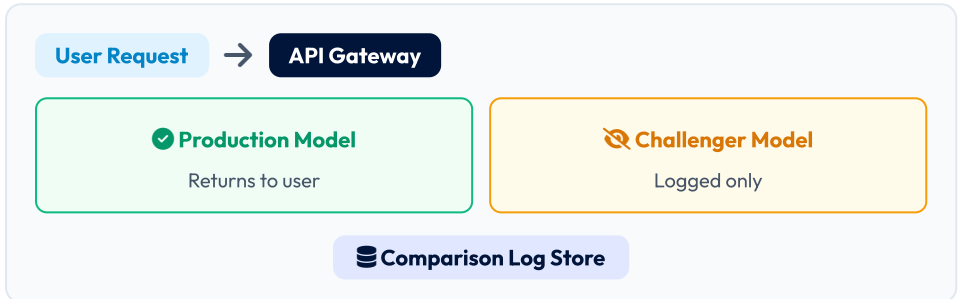
🔑 When to Use Each

Shadow for model replacement, prompt updates, RAG pipeline changes, or any change with unknown quality impact. Canary after shadow passes all quality gates.



Shadow Deployment Architecture

The shadow pipeline runs as an async background task. The user receives the production response immediately. The shadow response is logged for analysis without affecting user-facing latency.



⚡ Key Constraint

Shadow must NEVER block the user response path. It runs as a fire-and-forget async task. Any latency in the challenger model is invisible to the user.



Async Shadow Pipeline Implementation

```
import asyncio
from fastapi import FastAPI

app = FastAPI()

async def run_shadow_and_log(query: str, prod_response: str):
    try:
        shadow_resp = await asyncio.wait_for(
            challenger_pipeline(query), timeout=5.0 # hard timeout
        )
        await log_shadow_comparison({
            "query": query,
            "prod": prod_response,
            "shadow": shadow_resp,
            "ts": datetime.utcnow().isoformat()
        })
    except asyncio.TimeoutError:
        log_metric("shadow.timeout", 1)
    except Exception as e:
        log_metric("shadow.error", 1)

@app.post("/query")
async def handle_query(query: str):
    prod_resp = await prod_pipeline(query)
    asyncio.create_task(run_shadow_and_log(query, prod_resp))
    return {"answer": prod_resp} # returned instantly
```

- ! The 5-second timeout on the shadow call prevents resource leaks from slow challenger models. Errors in shadow must never propagate to the user.



Automated Comparison Metrics

```
from sentence_transformers import SentenceTransformer, util

enc = SentenceTransformer("BAAI/bge-small-en-v1.5")

def compare_responses(query, prod_resp, shadow_resp) → dict:
    p_emb = enc.encode([prod_resp], normalize_embeddings=True)
    s_emb = enc.encode([shadow_resp], normalize_embeddings=True)
    semantic_sim = float(util.dot_score(p_emb, s_emb)[0][0])
    len_ratio = len(shadow_resp.split()) / max(len(prod_resp.split()), 1)
    return {
        "semantic_similarity": semantic_sim,
        "length_ratio": len_ratio,
        "semantically_close": semantic_sim > 0.80,
        "length_ok": 0.5 ≤ len_ratio ≤ 2.0,
    }
```

Semantic Similarity

Above 0.90: very similar. 0.80 to 0.90: acceptable. Below 0.80: flag for human review. Below 0.60: potential regression.

Length Ratio

Shadow above 2x prod length may be verbose. Below 0.5x may be truncated. Both signal potential quality issues in the challenger.



Using GPT-4o as Automated Judge

For a sample of shadow comparisons, use GPT-4o to evaluate which response is better. Blind both model labels so the judge cannot be biased by which model produced which answer.

```
async def llm_judge_comparison(query, resp_a, resp_b) → dict:
    prompt = f"""You are evaluating two AI responses. Select the better one.

    Question: {query}

    Response A:
    {resp_a}

    Response B:
    {resp_b}

    Which response is more accurate, complete, and helpful?
    Reply with JSON: {"winner": "A" or "B", "reason": " ... "}}}"""

    result = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"}
    )
    return json.loads(result.choices[0].message.content)
```

 Run LLM-as-judge on 100 shadow pairs per day. Track challenger win rate. Must be above 0.45 (challenger not worse than prod) to proceed with canary.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Deterministic Canary Routing with MD5

Use a deterministic hash of the user ID to assign users to model versions. This ensures the same user always sees the same model, preventing confusing mid-conversation model switches.

```
import hashlib
import redis

r = redis.Redis()

def get_pipeline_version(user_id: str) → str:
    # Read live canary_pct from Redis (enables instant rollback)
    canary_pct = int(r.get("canary:pct") or 0)
    if canary_pct == 0:
        return "production"

    h = int(hashlib.md5(user_id.encode()).hexdigest(), 16)
    bucket = h % 100 # deterministic 0-99
    return "challenger" if bucket < canary_pct else "production"

# Ramp: r.set("canary:pct", 5) → 5% canary
# Rollback: r.set("canary:pct", 0) → instant 0% in under 1 second
```

Why Not Random

Random routing gives a different model to the same user on each request. Hashing ensures stable assignment. Users experience one consistent model throughout a conversation.



Automated Rollout Decision Logic

After 3 days of shadow testing, evaluate automated criteria before proceeding to canary. Never rely on intuition alone. All gates must pass.

```
def make_rollout_decision(shadow_metrics: dict) → tuple[str, dict]:
    prod_p99 = get_prod_p99_latency_ms()
    checks = {
        "semantic_ok": shadow_metrics["avg_similarity"] ≥ 0.80,
        "latency_ok": shadow_metrics["shadow_p99_ms"] ≤ 1.5 * prod_p99,
        "judge_ok": shadow_metrics["llm_win_rate"] ≥ 0.45,
        "error_ok": shadow_metrics["error_rate"] < 0.02,
        "refusal_ok": shadow_metrics["refusal_rate_delta"] < 0.05,
        "volume_ok": shadow_metrics["total_compared"] ≥ 1000,
    }
    action = "PROCEED_CANARY" if all(checks.values()) else "HOLD"
    return action, checks
```



Minimum 1,000 shadow comparisons before any canary decision. Fewer comparisons mean unreliable statistics.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

Safe Canary Ramp Schedule

Stage	Traffic	Monitor	Gate to Proceed
Shadow (3 days)	0% live	All shadow metrics	All 6 gates pass
Canary Day 1	1%	Error rate, latency p99	Both below baseline
Canary Day 3	5%	+ User feedback signals	No negative spike
Canary Day 7	20%	+ Cost per query, quality	LLM judge win rate above 0.45
Canary Day 14	50%	All production metrics	All gates green for 72h
Full Rollout	100%	Standard monitoring	Stable at 50% for 7 days

⏸ Hold at any stage if a gate fails. Investigate root cause before proceeding. Moving too fast has burned many production AI teams.



AI Coach John

Follow



↺ Repost

Prometheus Metrics for Shadow Comparison

```
from prometheus_client import Counter, Histogram, Gauge

shadow_similarity = Histogram(
    "shadow_semantic_similarity",
    "Cosine similarity of prod vs shadow",
    buckets=[0.5, 0.6, 0.7, 0.8, 0.85, 0.9, 0.95, 1.0]
)
shadow_errors = Counter(
    "shadow_errors_total", "Shadow pipeline errors",
    ["error_type"]
)
canary_win_rate = Gauge(
    "canary_llm_judge_win_rate", "Rolling win rate of challenger"
)

# Record every comparison
def record_shadow_metrics(comparison: dict):
    shadow_similarity.observe(comparison["semantic_similarity"])
    if comparison.get("judge_result"):
        win = comparison["judge_result"]["winner"] = "B"
        canary_win_rate.set(rolling_win_rate(win))
```

Grafana Dashboard

Build a Grafana panel with: similarity distribution histogram, daily judge win rate trend, shadow error rate, canary traffic percentage. Use 1-hour rolling windows for all metrics.



Instant Rollback via Redis Feature Flag

When a canary shows problems, you need to rollback in seconds not minutes. Storing the canary percentage in Redis means a single command reverts all traffic instantly without redeployment.

```
import redis

r = redis.Redis(decode_responses=True)

# Normal operations: adjust canary percentage
r.set("canary:pct", 5)    # 5% canary
r.set("canary:pct", 20)   # ramp to 20%
r.set("canary:pct", 50)   # ramp to 50%

# Instant rollback - all new requests go to production
r.set("canary:pct", 0)

# Automatic rollback when latency spikes
def auto_rollback_check():
    canary_p99 = get_canary_p99()
    prod_p99   = get_prod_p99()
    if canary_p99 > 2.0 * prod_p99:
        r.set("canary:pct", 0)
        alert_team("AUTO ROLLBACK: canary latency too high")
```



Redis set takes under 1ms. All new requests immediately route to production. No code deploy. No Docker build. No downtime.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Shadow and Canary Deployment Mistakes

- ✗ **Shadow that blocks the user response.** Any await on shadow results adds latency to the user. Shadow must be fire-and-forget using `asyncio.create_task`.
- ✗ **Random routing instead of hashed routing.** Random means the same user gets a different model on each request. Use `MD5(user_id) % 100` for stable assignment.
- ✗ **Not logging the exact prompt sent to each model.** Without the actual prompt text logged, you cannot debug why responses diverged between prod and challenger.
- ✗ **Fewer than 1,000 shadow comparisons before canary decision.** Quality differences are subtle. You need statistical significance before acting.
- ✗ **Skipping shadow for "small" prompt changes.** Even temperature or wording changes can cause large behavioral shifts. Always shadow test every change.
- ✗ **No automated rollback trigger.** Engineers may miss a spike at 3am. Build automated rollback based on latency and error rate thresholds.



Shadow and Canary **Full Reference**

Area	Decision
Shadow implementation	asyncio.create_task (fire-and-forget, 5s timeout)
Canary assignment	MD5(user_id) % 100, NOT random per request
Rollback mechanism	Redis canary:pct = 0, instant, no redeploy needed
Min shadow volume	1,000 comparisons before any go/no-go decision
Quality gate 1	Semantic similarity avg above 0.80
Quality gate 2	LLM judge win rate above 0.45
Latency gate	Shadow p99 not more than 1.5x production p99
Ramp schedule	0% shadow then 1% then 5% then 20% then 50% then 100%

 **Shadow = zero user risk. Canary = controlled exposure. Never skip either stage.**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

 KEY TAKEAWAY

Never Deploy AI Changes Directly to Production

What feels like a small prompt edit can silently change your model's behavior for thousands of users. Shadow and canary give you a safety net every single time.

✓ Your 3 Starting Steps

- Wrap your LLM call in a fire-and-forget shadow task using `asyncio.create_task`
- Store `canary_pct` in Redis so you can ramp and rollback without redeployment
- Add semantic similarity logging to every shadow comparison from day one

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**